Copyright All rights reserved

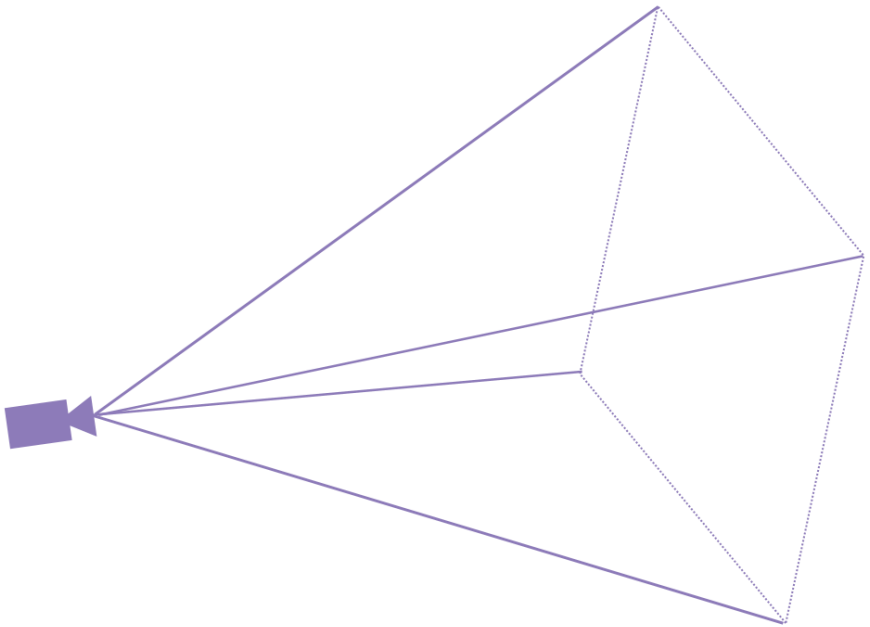
12. 원근 투영

화면에 현실감을 부여하는 변환

12.1 원근 투영 변환의 원리

Heesung Oh
Copyright All rights reserved

- 투시 원근법
  - 3차원 공간 → 2차원 화면으로 표현하는 방법
  - 거리(깊이)에 따라 크기 변화 발생
  - 가까울수록 크게, 멀수록 작게 보임



- 직교 투영 (Orthographic)
  - 크기 변화 없음
  - 평행선 유지

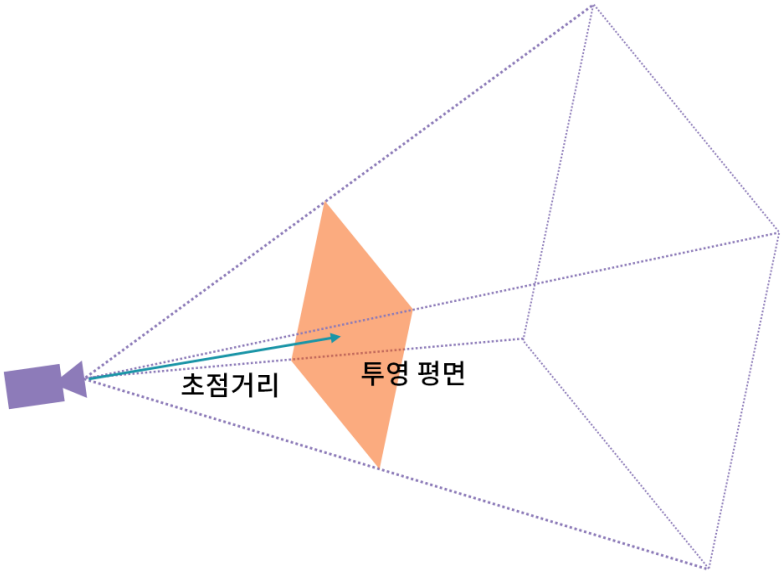
- 원근 투영 (Perspective)
  - 시점 기준 3차원 점을
    깊이(z)에 따라 축소하여 2차원 평면에 사상하는 변환

- 화각 (Field of View, FOV)
  - 카메라가 한 번에 볼 수 있는 시야의 각도
  - 값이 클수록 넓게 보이고, 작을수록 좁게 보임

12.1 원근 투영 변환의 원리

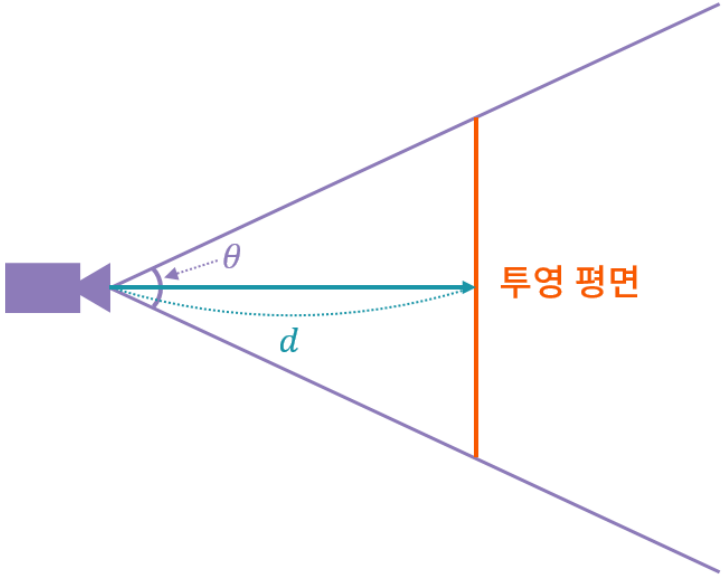
Heesung Oh
Copyright All rights reserved

- 투영 평면
  - 3D 공간의 점을 투영해서 그려지는 카메라 앞의 가상 화면
  - 보통 Near Plane



- 초점 거리 (Focal Length)
  - 카메라에서 투영 평면까지 거리

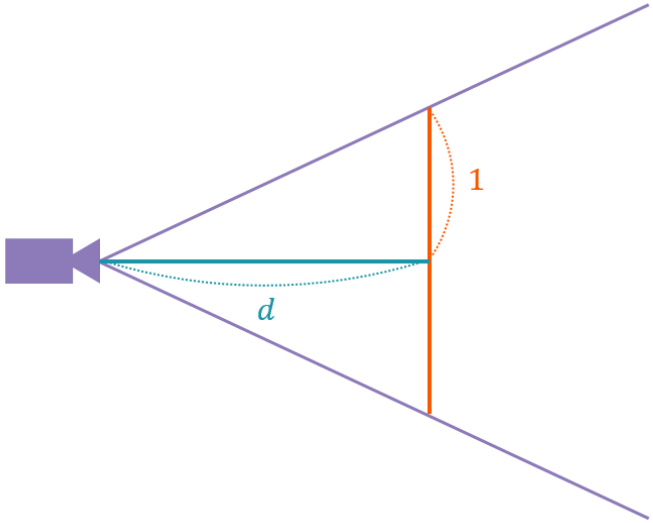
- 측면에서 바라본 투영 공간과 투영 평면

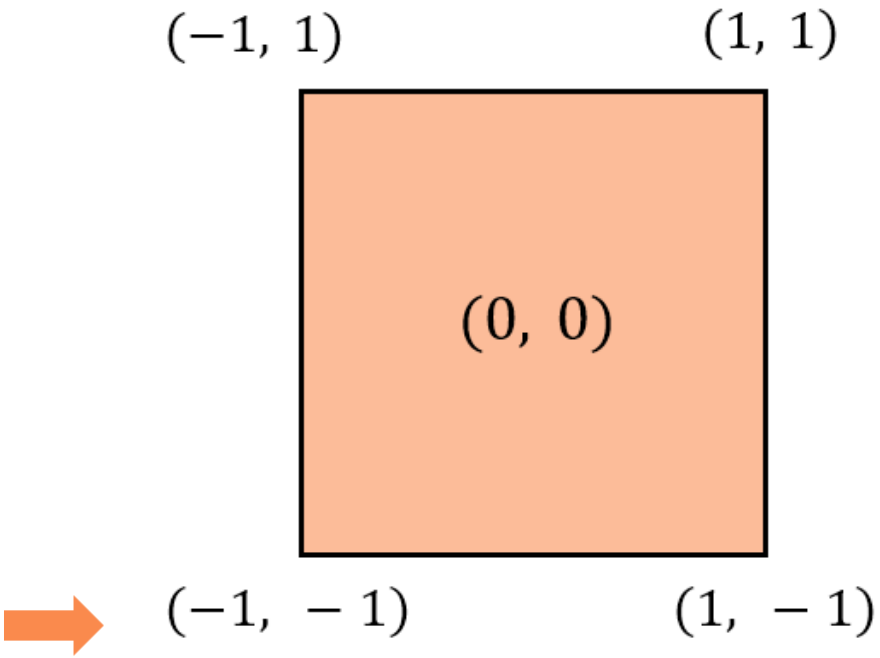


Heesung Oh
Copyright All rights reserved

- NDC 영역 (Normalized Device Coordinates)
  - 투영 변환 + Perspective Divide 이후의 좌표 공간
  - 화면 매핑 전의 정규화 좌표

- 좌표 범위
  - x∈[-1, +1], y∈[-1, +1]
  - z∈[0, +1] (DirectX)
  - z∈[-1, +1] (OpenGL)



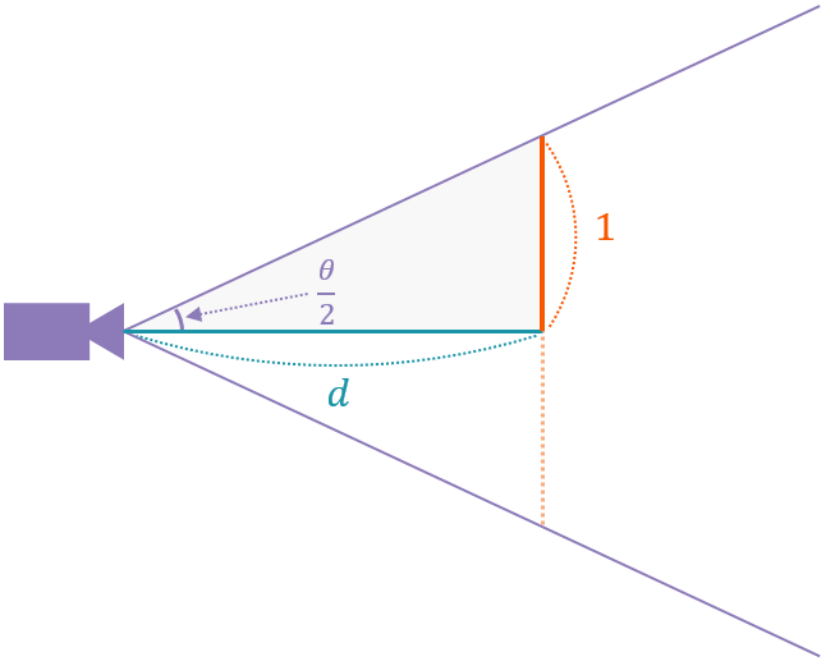


Heesung Oh
Copyright All rights reserved

- **화각과 초점 거리의 관계**

$$\tan\left(\frac{\theta}{2}\right) = \frac{1}{d}$$

$$d = \frac{1}{\tan\left(\frac{\theta}{2}\right)}$$



12.1 원근 투영 변환의 원리

Heesung Oh
Copyright All rights reserved

- 유클리드 공간 (Euclidean Space)
  - 거리와 각도가 정의되는 공간
  - 거리·각도 보존, 직선 최단 경로

- 사영 공간 (Projective Space)
  - 원근 투영을 표현하는 공간
    → 원근 투영을 수학적으로 표현하는 공간

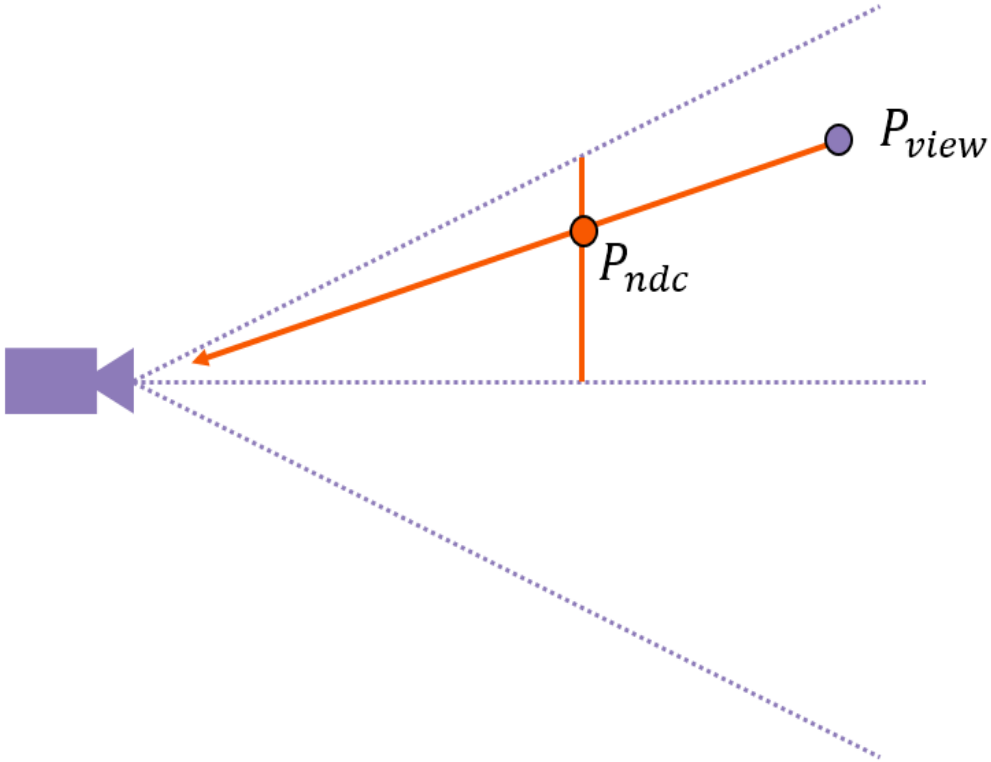
  - 평행선이 한 점에서 만난다고 가정
  - 길이·각도는 보존되지 않음, 비율만 유지

Heesung Oh
Copyright All rights reserved

- **투영의 진행 과정**

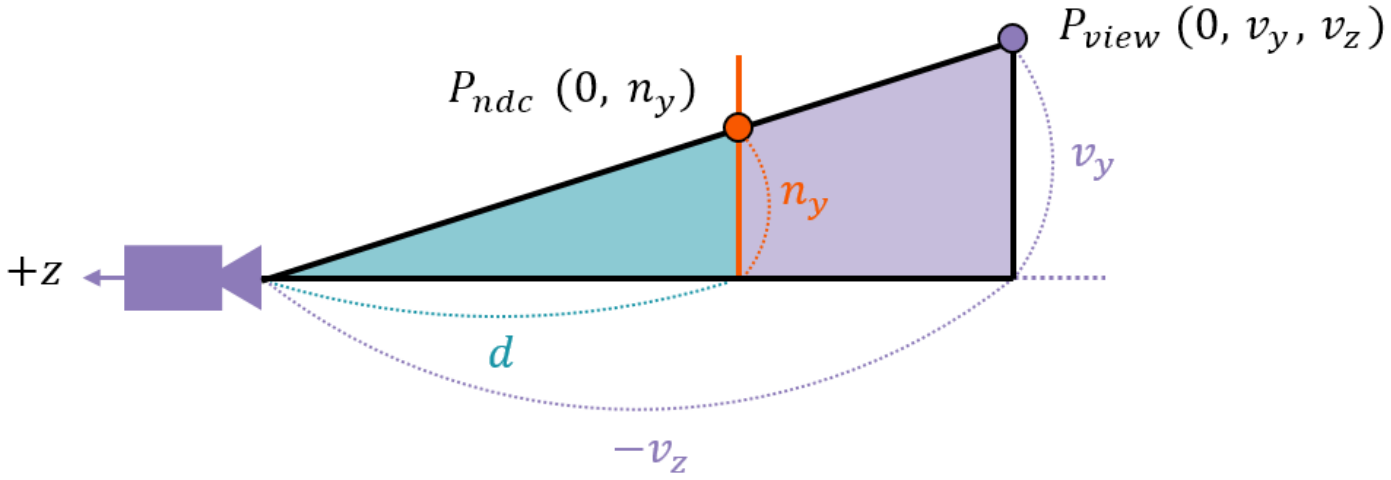
$$P_{view} = (0, v_y, v_z)$$

$$P_{ndc} = (0, n_y)$$



Heesung Oh
Copyright All rights reserved

- **두 개의 닮은꼴 삼각형**



$$n_y : d = v_y : -v_z$$

$$n_y = \frac{d \cdot v_y}{-v_z}$$

$$n_x = \frac{d \cdot v_x}{-v_z}$$

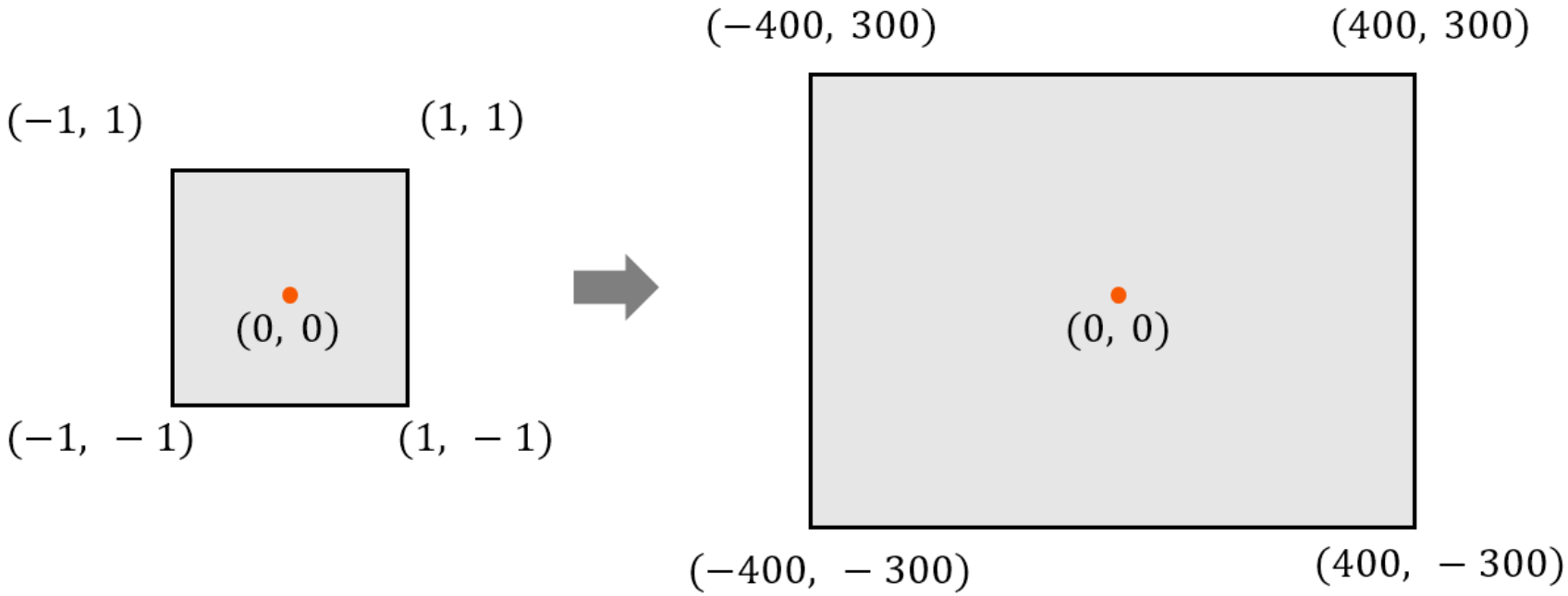
$$P_{ndc} = (n_x, n_y)$$

$$= \left(\frac{d \cdot v_x}{-v_z}, \frac{d \cdot v_y}{-v_z}\right)$$

$$= -\frac{d}{v_z}(v_x, v_y)$$

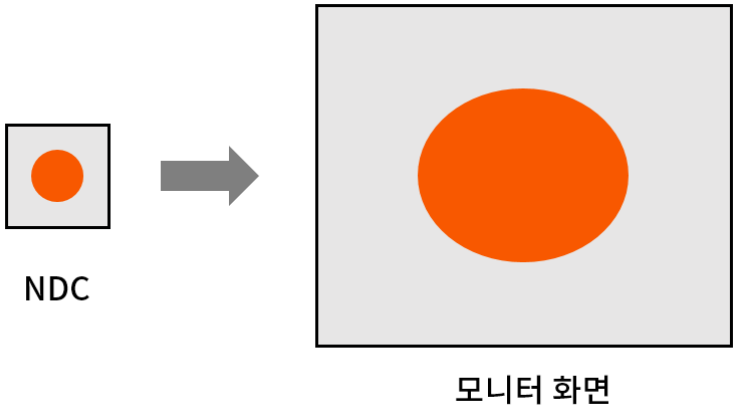
Heesung Oh
Copyright All rights reserved

- NDC에서 화면 공간으로의 변환
  - ex) NDC → 800 X 600

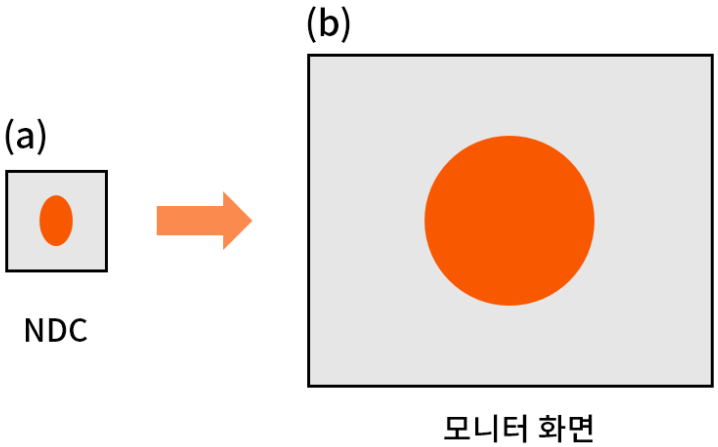


Heesung Oh
Copyright All rights reserved

- 종횡비를 고려하지 않고 그대로 확대하는 경우의 문제



- 종횡비(Aspect ratio) 적용



Heesung Oh
Copyright All rights reserved

- **종횡비(Aspect ratio) 적용**

$$P_{ndc} = -\frac{d}{v_z}(v_x, v_y) \rightarrow P_{ndc} = -\frac{d}{v_z}\left(\frac{v_x}{a}, v_y\right)$$

$$P_{ndc} = P \cdot \vec{v}$$

$$= \begin{bmatrix} \frac{1}{a} \cdot \frac{d}{-v_z} & 0 \\ 0 & \frac{d}{-v_z} \end{bmatrix} \cdot \begin{bmatrix} v_x \\ v_y \end{bmatrix}$$

$$= \begin{bmatrix} \frac{d}{a} & 0 & 0 \\ 0 & d & 0 \\ 0 & 0 & -1 \end{bmatrix} \begin{bmatrix} v_x \\ v_y \\ v_z \end{bmatrix} = \begin{bmatrix} \frac{d}{a} \cdot v_x \\ d \cdot v_y \\ -v_z \end{bmatrix}$$

Heesung Oh
Copyright All rights reserved

- 클립 좌표(Clip Coordinate)
  - 원근 투영 행렬 P로 변환되는 좌표계

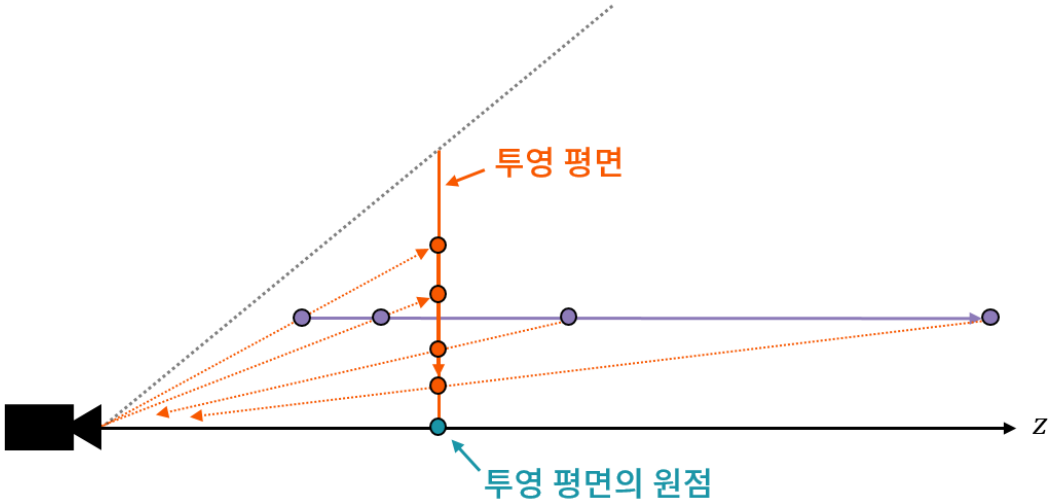
$$P_{clip} = \left(\frac{d \cdot v_x}{a}, d \cdot v_y, -v_z \right)$$

- 클립 좌표(Clip Coordinate) → NDC 좌표
  - 마지막 $-v_z$ 로 나눔

$$P_{ndc} = \left(\frac{d \cdot v_x}{-a \cdot v_z}, \frac{d \cdot v_y}{-v_z}, 1 \right)$$

Heesung Oh
Copyright All rights reserved

- 사영 공간 내에서 평행 이동하는 점에 따른 투영 결과



$$x = \frac{x'}{z'} \qquad y = \frac{y'}{z'}$$

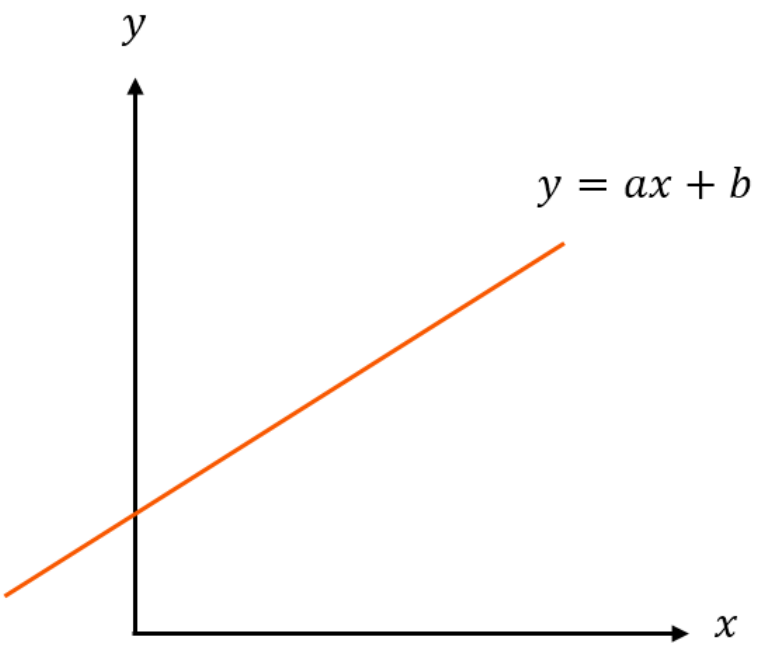
Heesung Oh
Copyright All rights reserved

- **NDC 좌표에서의 직선 방정식**

$$y = ax + b$$

$$\frac{y'}{z'} = a\frac{x'}{z'} + b$$

$$y' = ax' + bz'$$



- **동차 방정식**
  - 모든 항이 같은 차수로 이루어진 방정식
    → 미지수에 대한 차수가 모두 동일
  - 스케일을 곱해도 형태가 변하지 않음

12.2 동차 좌표계

Heesung Oh
Copyright All rights reserved

- 소실점 (Vanishing Point)
  - 평행한 직선들이 원근 투영에서 한 점으로 모여 보이는 점
  - 3차원 방향 정보가 2차원으로 투영되며 생김

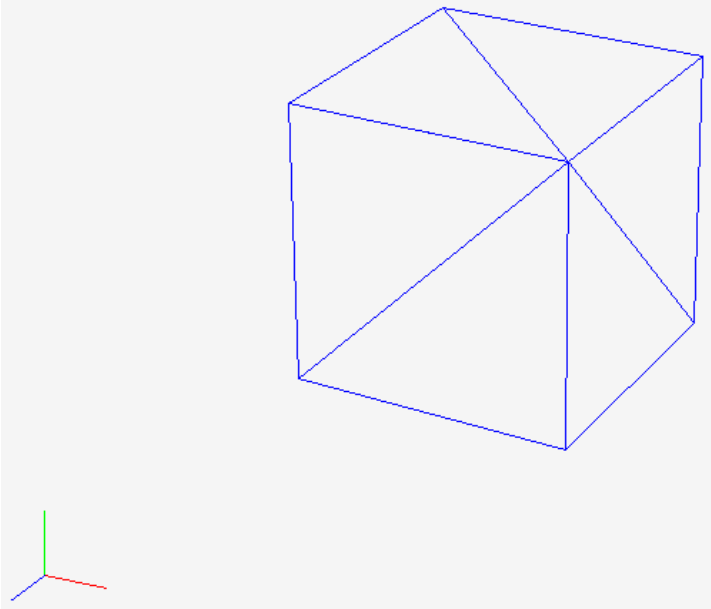
- 미델하르니스의 가로수길

실습 12-1 화면에 원근감 부여하기

Heesung Oh
Copyright All rights reserved

- **원근 투영 행렬**

$$P = \begin{bmatrix} \frac{d}{a} & 0 & 0 & 0 \\ 0 & d & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

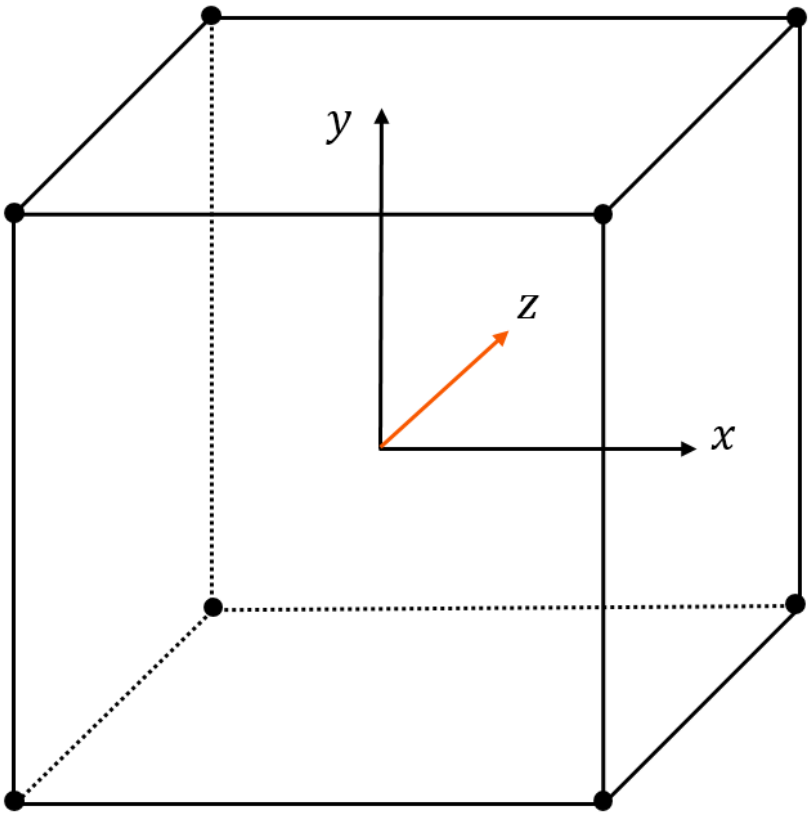


```
Matrix4x4 CameraObject::GetPerspectiveMatrix() const {

  float invA = 1.f / _ViewportSize.AspectRatio();
  float d = 1.f / tanf(Math::Deg2Rad(_FOV) * 0.5f);

  return Matrix4x4(
        Vector4::UnitX * invA * d,
        Vector4::UnitY        * d,
        Vector4(0.f, 0.f, -1.f, 0.f),
        Vector4(0.f, 0.f,    0, 1.f)
  );
}
```

```
// 클립 좌표를 NDC 좌표로 변경
// 무한 원점인 경우, 약간 보정해준다.
if (v.Position.Z == 0.f)
    v.Position.Z = SMALL_NUMBER;

float invZ = 1.f / v.Position.Z;
v.Position.X *= invZ;
v.Position.Y *= invZ;
v.Position.Z *= invZ;
```

Heesung Oh
Copyright All rights reserved

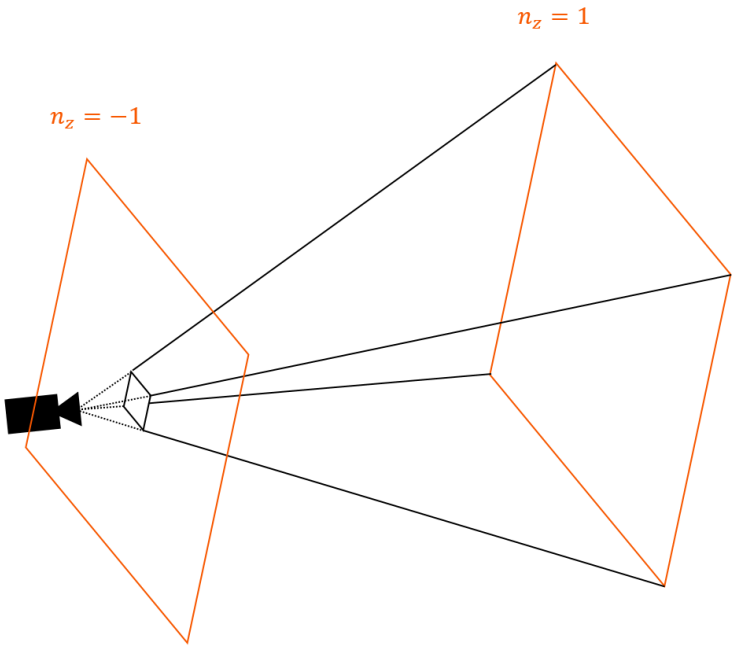
- 3차원으로 확장된 NDC 영역



12.3 깊이 값

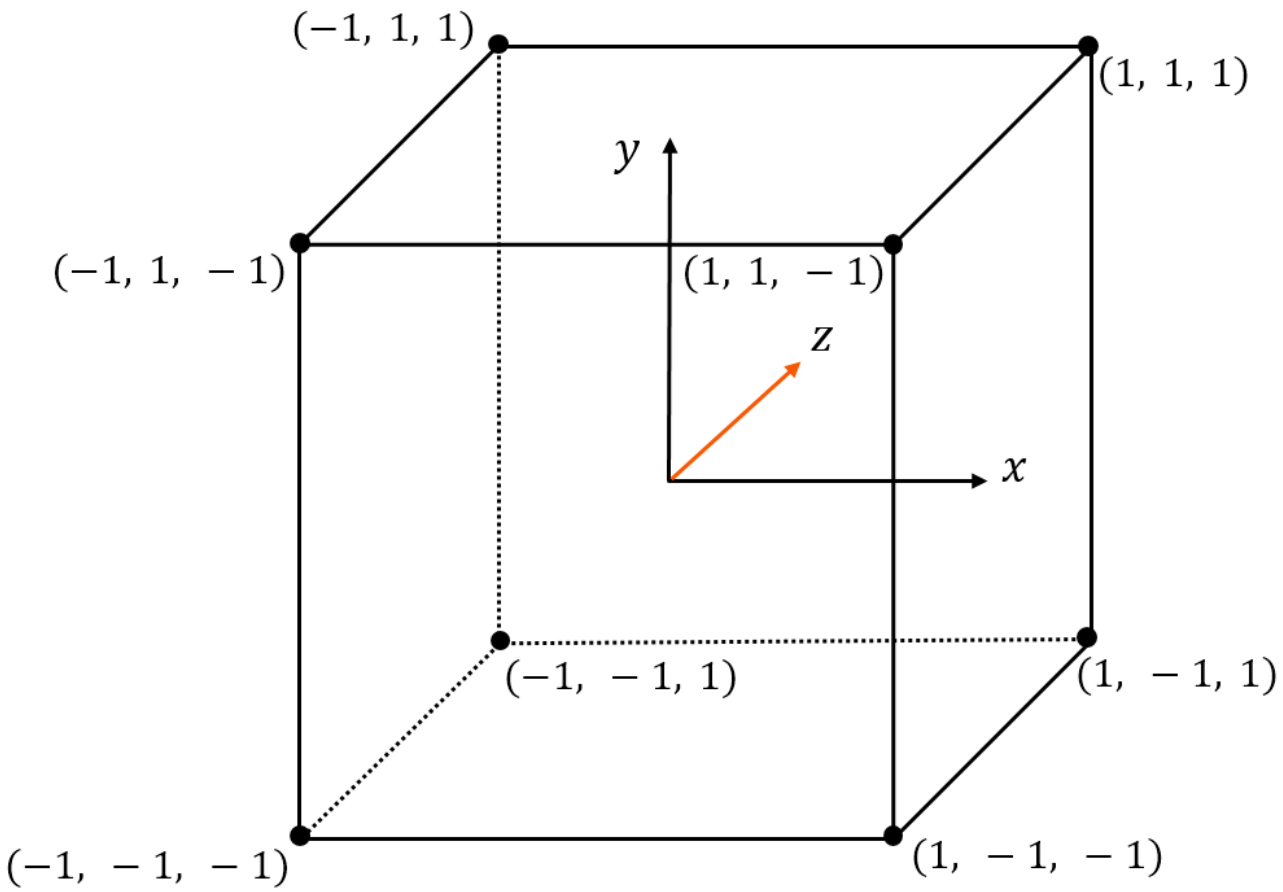
Heesung Oh
Copyright All rights reserved

- 근 평면 (Near plane)
  - 카메라에서 가장 가까운 투영 기준 평면
  - 깊이 계산의 시작 기준
- 원 평면 (Far plane)
  - 카메라에서 가장 먼 투영 제한 평면
  - 깊이 계산의 끝 기준
- 절두체 (Frustum)
  - 원근 투영의 대상 공간
  - 근 평면과 원 평면 사이의 잘린 피라미드 형태의 공간
  - 카메라가 실제로 볼 수 있는 영역(View Volume)
  - 이 영역 안의 객체만 렌더링 → 밖에 있는 객체는 클리핑



Heesung Oh
Copyright All rights reserved

- 3차원으로 확장한 NDC의 범위



Heesung Oh
Copyright All rights reserved

- 3차원으로 확장한 NDC의 범위

$$P \cdot \vec{v} = \begin{bmatrix} \frac{d}{a} & 0 & 0 & 0 \\ 0 & d & 0 & 0 \\ i & j & k & l \\ 0 & 0 & -1 & 0 \end{bmatrix} \begin{bmatrix} v_x \\ v_y \\ v_z \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{d}{a} \cdot v_x \\ d \cdot v_y \\ ? \\ -v_z \end{bmatrix}$$

- 깊이 값은 뷰 좌표계 의 x, y 축에 직교 ➔ i, j → 0
- Near 평면 $(0,0,-n,1) \rightarrow (0,0,-1)$
- Far 평면 $(0,0,f,1) \rightarrow (0,0,1)$

12.3 깊이 값

Heesung Oh
Copyright All rights reserved

- **Near 평면** $(0, 0, -n, 1) \rightarrow (0, 0, -1)$
- **Far 평면** $(0, 0, \ f, 1) \rightarrow (0, 0, \ 1)$

$$P_1 = \begin{bmatrix} \frac{d}{a} & 0 & 0 & 0 \\ 0 & d & 0 & 0 \\ 0 & 0 & k & l \\ 0 & 0 & -1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ -n \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ ? \\ n \end{bmatrix}$$

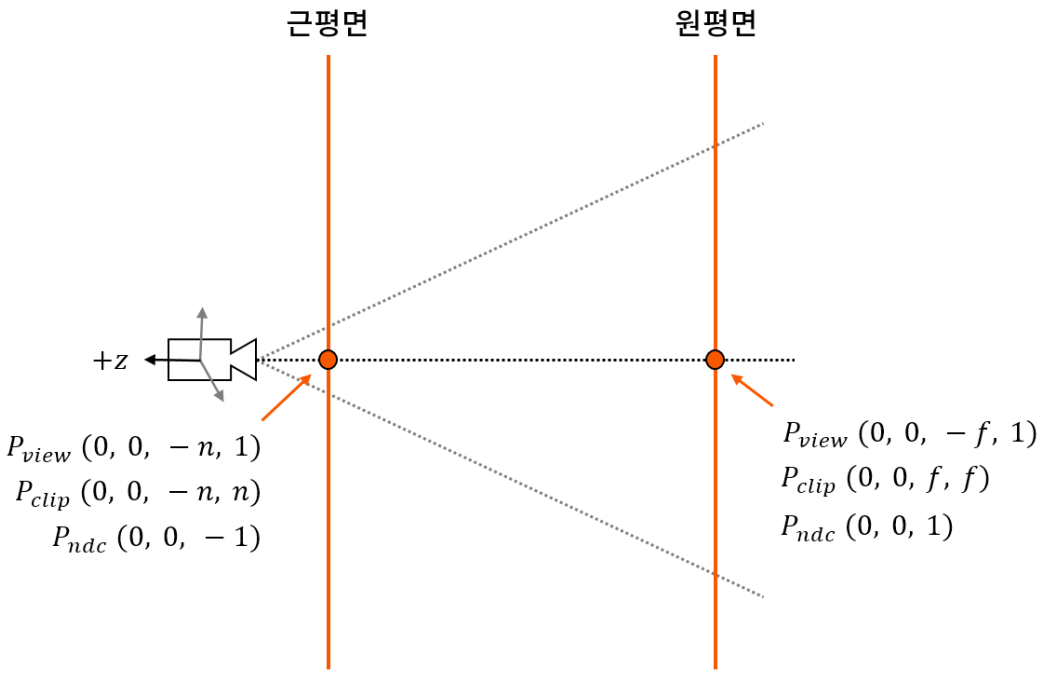
$$P_2 = \begin{bmatrix} \frac{d}{a} & 0 & 0 & 0 \\ 0 & d & 0 & 0 \\ 0 & 0 & k & l \\ 0 & 0 & -1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ -f \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ ? \\ f \end{bmatrix}$$

Heesung Oh
Copyright All rights reserved

투영 행렬

$$P_1 = \begin{bmatrix} \frac{d}{a} & 0 & 0 & 0 \\ 0 & d & 0 & 0 \\ 0 & 0 & k & l \\ 0 & 0 & -1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ -n \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ -n \\ n \end{bmatrix}$$

$$P_2 = \begin{bmatrix} \frac{d}{a} & 0 & 0 & 0 \\ 0 & d & 0 & 0 \\ 0 & 0 & k & l \\ 0 & 0 & -1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ -f \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ f \\ f \end{bmatrix}$$



Heesung Oh
Copyright All rights reserved

- **투영 행렬 (Projection matrix)**

$$-kn + l = -n$$
$$-kf + l = f$$

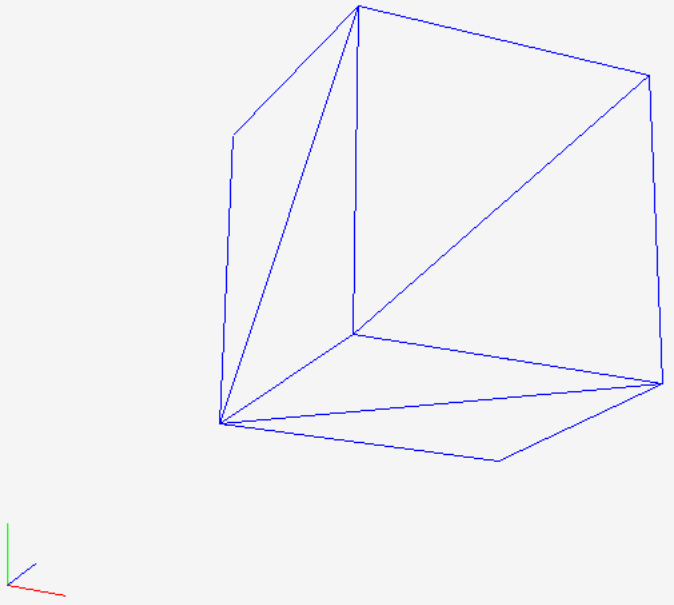
$$k = \frac{-n - f}{-n + f} = \frac{n + f}{n - f}$$

$$l = f + kf = f + \frac{n + f}{n - f} f$$
$$= \frac{fn - ff + nf + ff}{n - f}$$
$$l = \frac{2nf}{n - f}$$

$$P = \begin{bmatrix} \frac{d}{a} & 0 & 0 & 0 \\ 0 & d & 0 & 0 \\ 0 & 0 & \frac{n+f}{n-f} & \frac{2nf}{n-f} \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

Heesung Oh
Copyright All rights reserved

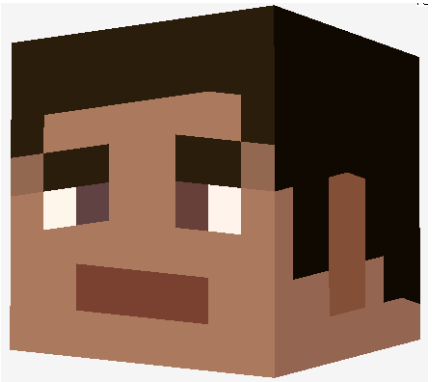
• 투영 행렬

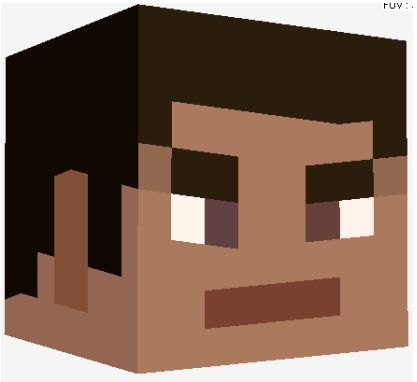


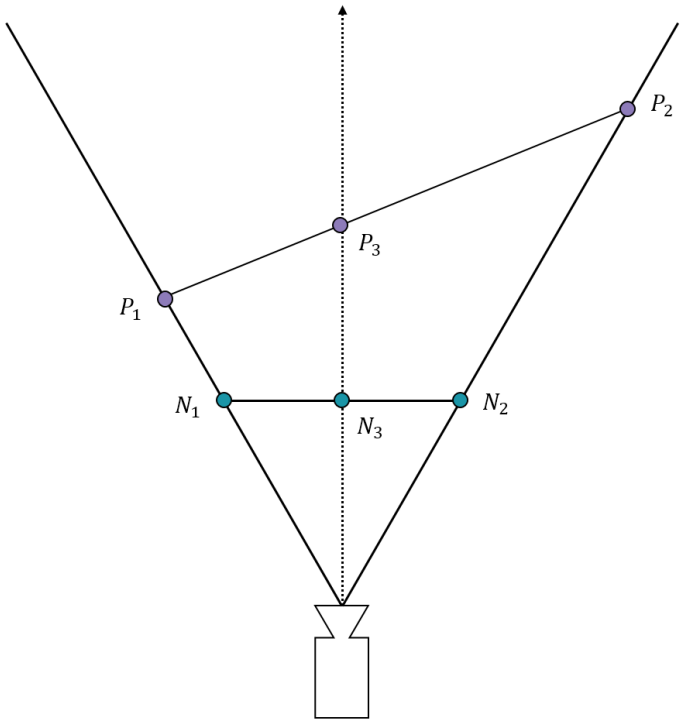
```
Matrix4x4 CameraObject::GetPerspectiveViewMatrix() const {
    // 뷰 행렬 관련 요소
    Vector3 viewX, viewY, viewZ;
    GetViewAxes(viewX, viewY, viewZ);
    Vector3 pos = _Transform.GetPosition();
    float zPos = viewZ.Dot(pos);

    // 투영 행렬 관련 요소
    float invA = 1.f / _ViewportSize.AspectRatio();
    float d = 1.f / tanf(Math::Deg2Rad(_FOV) * 0.5f);
    float dx = invA * d;
    float invNF = 1.f / (_NearZ - _FarZ);
    float k = (_FarZ + _NearZ) * invNF;
    float l = 2.f * _FarZ * _NearZ * invNF;

    return Matrix4x4(
    Vector4(dx * viewX.X, d * viewY.X, k * viewZ.X, -viewZ.X),
    Vector4(dx * viewX.Y, d * viewY.Y, k * viewZ.Y, -viewZ.Y),
    Vector4(dx * viewX.Z, d * viewY.Z, k * viewZ.Z, -viewZ.Z),
    Vector4(-dx * viewX.Dot(pos), -d * viewY.Dot(pos), -k * zPos + l, zPos)
    );
}
```

Heesung Oh
Copyright All rights reserved

- 시야각에 걸쳐 있는 두 점과 이를 투영한 결과





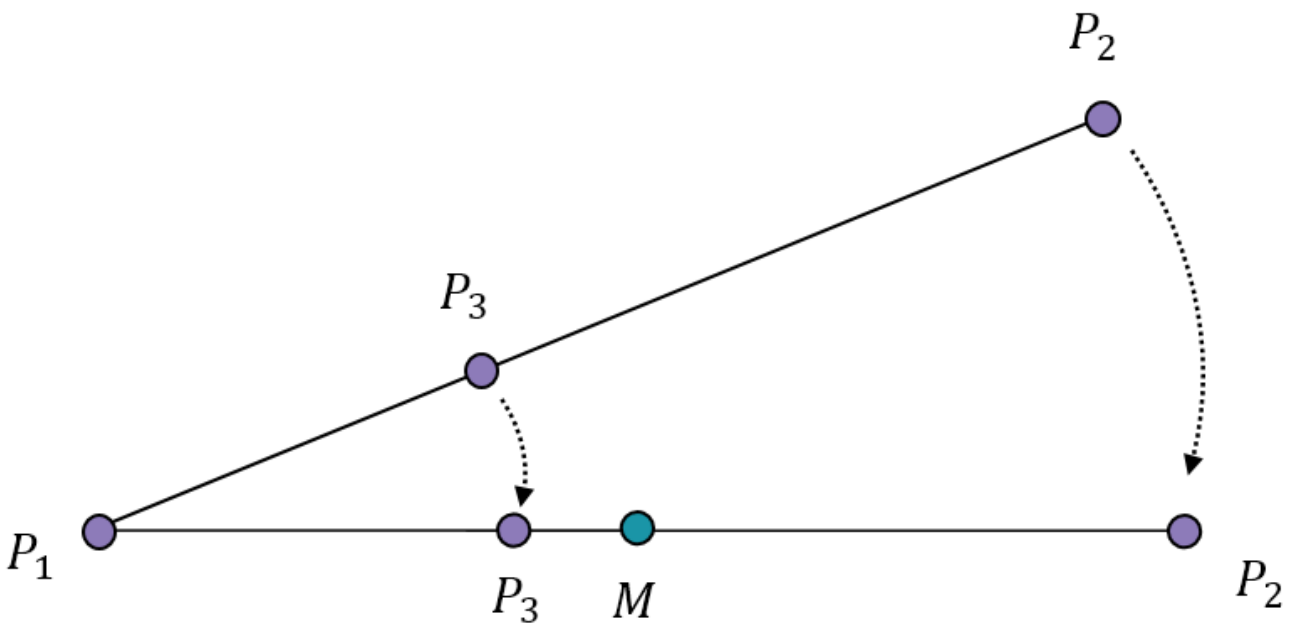


12.4 원근 보정 매핑

Heesung Oh
Copyright All rights reserved

- 뷰 공간의 선을 수평으로 조정해 비교한 결과
  - NDC 의 무게 중심 좌표 ≠ 사영 공간 무게 중심 좌표

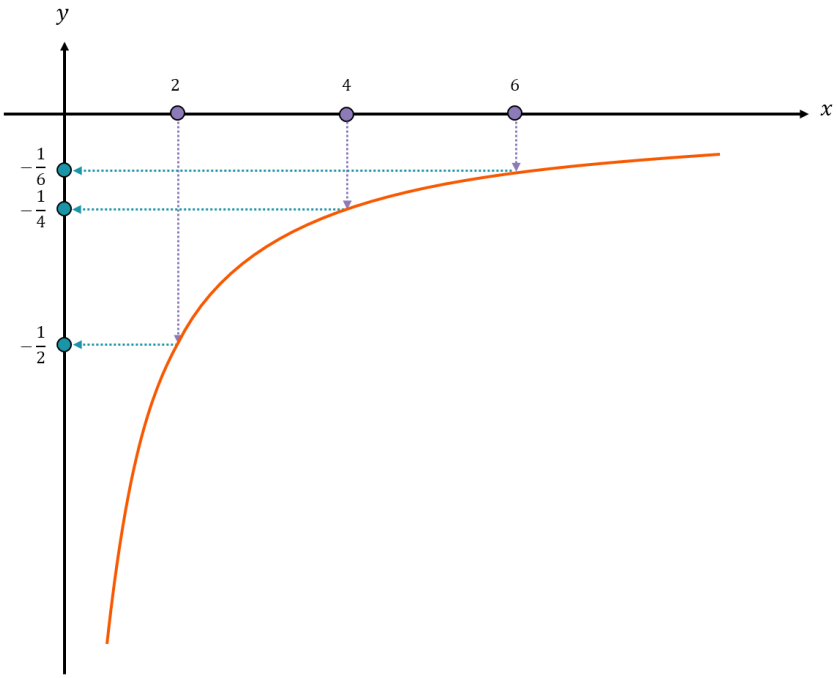
- 투영 보정 보간 (Perspective correction interpoliation)
  - NDC 무게 중심 좌표 → 사영 공간 무게 중심 좌표 계산
  - 투영 전 무게 중심 좌표 값 → 투영 보정 보간



12.4 원근 보정 매핑

Heesung Oh
Copyright All rights reserved

- $y = -\frac{1}{x}$ 의 그래프



$$y' = q_1 \cdot y_1 + q_2 \cdot y_2$$

$$x' = t_1 \cdot x_1 + t_2 \cdot x_2$$

$$\frac{1}{x'} = q_1 \cdot \frac{1}{x_1} + q_2 \cdot \frac{1}{x_2}$$

Heesung Oh
Copyright All rights reserved

- **투영 보정 보간 (Perspective correction interpoliation)**

$$y' = q_1 \cdot y_1 + q_2 \cdot y_2$$

$$x' = t_1 \cdot x_1 + t_2 \cdot x_2$$

$$\frac{1}{x'} = q_1 \cdot \frac{1}{x_1} + q_2 \cdot \frac{1}{x_2}$$

$$x' = \frac{1}{q_1 \cdot \frac{1}{x_1} + q_2 \cdot \frac{1}{x_2}}$$

$$x' \left(q_1 \cdot \frac{1}{x_1} + q_2 \cdot \frac{1}{x_2} \right) = 1$$

$$q_1 \cdot \frac{x'}{x_1} + q_2 \cdot \frac{x'}{x_2} = 1$$

$$q_1 \cdot \frac{x'}{x_1} + q_2 \cdot \frac{x'}{x_2} = t_1 + t_2$$

$$t_1 = \frac{x'}{x_1} q_1 \qquad t_2 = \frac{x'}{x_2} q_2$$

12.4 원근 보정 매핑

Heesung Oh
Copyright All rights reserved

- 3점으로 확대

$$x' = \frac{1}{q_1 \cdot \frac{1}{x_1} + q_2 \cdot \frac{1}{x_2} + q_3 \cdot \frac{1}{x_3}}$$

$$t_1 = \frac{x'}{x_1} q_1 \quad t_2 = \frac{x'}{x_2} q_2 \quad t_3 = \frac{x'}{x_3} q_3$$

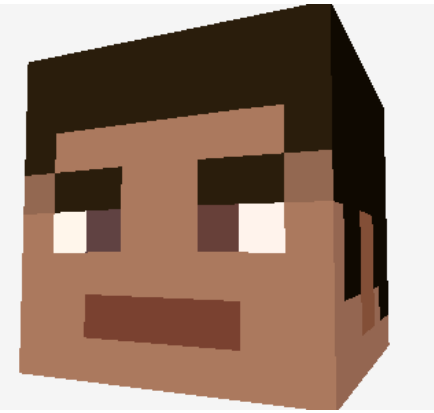
- x → −z 로 치환, 원근 보정 매핑 완성

$$z' = \frac{1}{q_1 \cdot \frac{1}{z_1} + q_2 \cdot \frac{1}{z_2} + q_3 \cdot \frac{1}{z_3}}$$

$$t_1 = \frac{z'}{z_1} q_1 \quad t_2 = \frac{z'}{z_2} q_2 \quad t_3 = \frac{z'}{z_3} q_3$$

Heesung Oh
Copyright All rights reserved

- 원근 보정 매핑



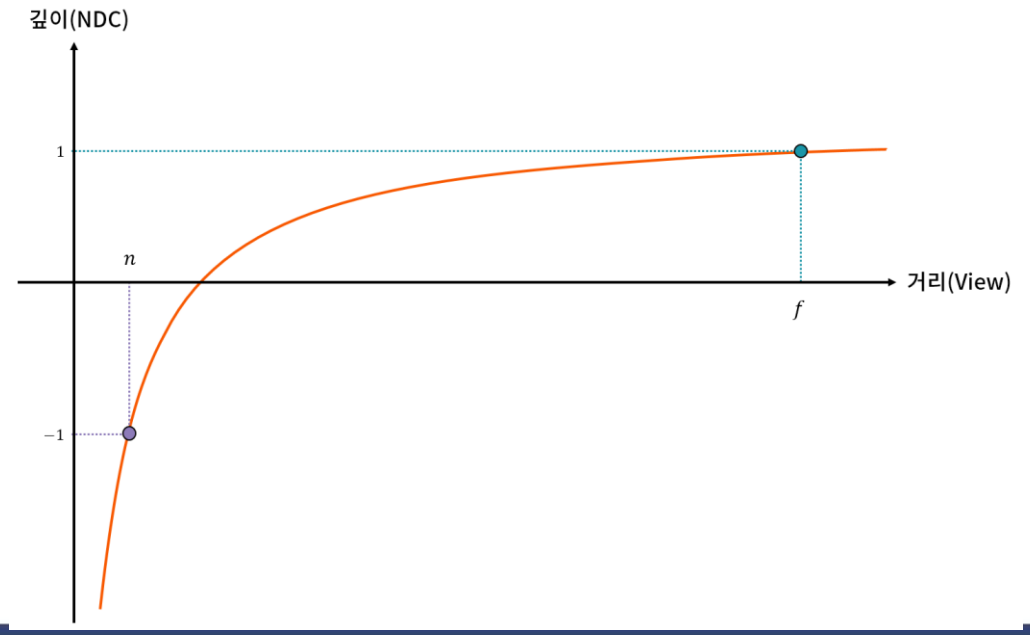
```
float s = (wdotv * udotv - wdotu * vdotv) * invDenominator;
float t = (wdotu * udotv - wdotv * udotu) * invDenominator;
float oneMinusST = 1.f - s - t;

if (((s >= 0.f) && (s <= 1.f)) && ((t >= 0.f) && (t <= 1.f))
                && ((oneMinusST >= 0.f) && (oneMinusST <= 1.f)))
{
  // 투영보정에 사용할 공통 분모
  float z = invZ0 * oneMinusST + invZ1 * s + invZ2 * t;
  float invZ = 1.f / z;

  Vector2 targetUV = (InVertices[0].UV * oneMinusST * invZ0
                  + InVertices[1].UV * s * invZ1 + InVertices[2].UV * t * invZ2) * invZ;

r.DrawPoint(fragment, FragmentShader3D(mainTexture.GetSample(targetUV), LinearColor::White));
```

12.5 깊이 버퍼

Heesung Oh
Copyright All rights reserved

- 깊이 버퍼 (Depth Buffer, Z-Buffer)
  - 픽셀마다 깊이(z) 값을 저장하는 버퍼
  - 더 가까운 픽셀만 화면에 표시, 어떤 물체가 앞에 보일지 결정
    - 기존 깊이 vs 새 깊이 비교
    - 더 가까우면 갱신, 아니면 버림
  - 절두체(Near~Far) 범위 깊이 사용

- 뷰 공간 좌표값 NDC 좌표값 비교

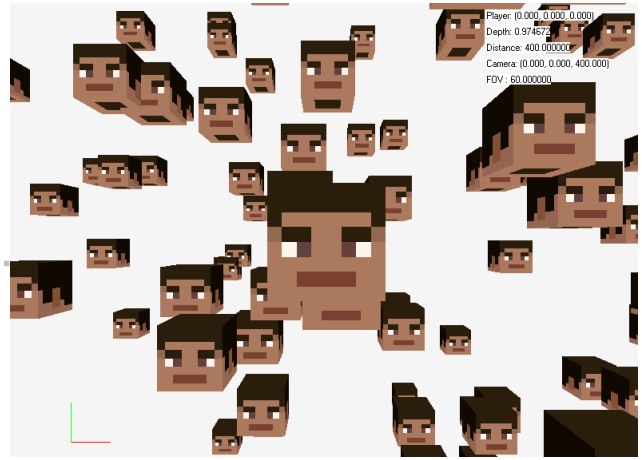


실습 12-4 깊이 버퍼 활용

Heesung Oh
Copyright All rights reserved

- 삼각형 3 점 z_1, z_2, z_3 의 깊이 값

$$z' = q_1 \cdot z_1 + q_2 \cdot z_2 + q_3 \cdot z_3$$




```
// 투영보정에 사용할 공통 분모
float z = invZ0 * oneMinusST + invZ1 * s + invZ2 * t;
float invZ = 1.f / z;

// 깊이 버퍼 테스팅
if (toggleDepthTesting)
{
  // 깊이 테스팅

float newDepth = InVertices[0].Position.Z * oneMinusST + InVertices[1].Position.Z
 * s
                    + InVertices[2].Position.Z * t;
  float prevDepth = r.GetDepthBufferValue(fragment);

  if (newDepth < prevDepth)
  {
    // 픽셀을 처리하기 전 깊이 값을 버퍼에 보관
    r.SetDepthBufferValue(fragment, newDepth);
  }
  else
  {
    // 이미 앞에 무언가 그려져있으므로 픽셀그리기는 생략
    continue;
```

실습 12-5 깊이 버퍼 시각화

Heesung Oh
Copyright All rights reserved

- 깊이 색상
  - n_z 로 하면 구분 안감

$$C_{depth} = (n_z + 1) \cdot 0.5$$

  - 뷰공간 깊이 값 사용

$$z' = \frac{1}{q_1 \cdot \frac{1}{z_1} + q_2 \cdot \frac{1}{z_2} + q_3 \cdot \frac{1}{z_3}} \qquad C_{depth} = \frac{z' - n}{f - n}$$

```
if (IsDepthBufferDrawing())
{
  float grayScale = (newDepth + 1.f) * 0.5f;
  if (useLinearVisualization)
  {
    // 카메라로부터의 거리에 따라 균일하게 증감하는 흑백 값으로 변환
    grayScale = (invZ - n) / (f - n);
  }

  // 뎁스 버퍼 그리기
  r.DrawPoint(fragment, LinearColor::White * grayScale);
```

12.6 정리

Heesung Oh
Copyright All rights reserved

- 원근 투영 행렬 (Perspective Projection Matrix)
  - 3D 좌표를 원근이 적용된 Clip Space로 변환
  - w로 나누어 원근 효과 생성
- 원근 보정 매핑 (Perspective-Correct Mapping)
  - 텍스처 좌표를 원근 왜곡 없이 보간
  - 1/w를 이용해 선형 보간 수행
- 깊이 버퍼 (Depth Buffer)
  - 픽셀마다 깊이 값을 저장하여 가시성 결정
  - 더 가까운 픽셀만 화면에 표시

Copyright All rights reserved

13. 절두체

최적화된 3차원 공간